

Proyecto de Curso: Análisis, Diseño y Construcción de un Sistema Operativo desde Cero

Por

Castro Pari, Rayneld Fidel

Mamani Flores, Natan

Mendoza Quispe, Jose Daniel

Polo Chura, Marco Rosauro

Trabajo académico presentado a la

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

como

Proyecto de Investigación de la Primera Unidad de Sistemas Operativos



Departamento Académico de Ingeniería de Informática y de Sistemas

Asesor: Ugarte Rojas, Héctor Eduardo

Memorial Universidad Nacional San Antonio Abad del Cusco

Semestre 2025-II

Cusco

Perú

Índice general

Índice de tablas	vi
Índice de figuras	vii
Introducción	1
1 Propuestas analizadas	4
1.1 MINIX	5
1.1.1 Nombre del proyecto o sistema operativo	5
1.1.2 Enlace al repositorio y/o documentación oficial	5
1.1.3 Objetivo del proyecto	5
1.1.4 Lenguaje(s) de implementación	6
1.1.5 Arquitectura del sistema (monolítica, microkernel, etc.)	6
1.1.6 Componentes implementados (procesos, memoria, archivos, etc.)	8
1.1.6.1 Sistema de archivos	9
1.1.6.2 Archivos especiales	10
1.1.6.3 Tuberías	10
1.1.6.4 Shell	10

1.1.6.5	Llamadas al sistema	10
1.1.6.6	Comunicación entre procesos	11
1.1.7	Herramientas utilizadas (compiladores, emuladores, etc.) . . .	11
1.1.8	Nivel de complejidad y accesibilidad para estudiantes	12
1.2	Theseus	13
1.2.1	Nombre del proyecto o sistema operativo	13
1.2.2	Enlace al repositorio y/o documentación oficial	13
1.2.3	Objetivo del proyecto	13
1.2.4	Lenguaje(s) de implementación	13
1.2.5	Arquitectura del sistema (monolítica, microkernel, etc.)	14
1.2.6	Componentes implementados (procesos, memoria, archivos, etc.)	16
1.2.7	Herramientas utilizadas (compiladores, emuladores, etc.) . . .	19
1.2.8	Nivel de complejidad y accesibilidad para estudiantes	20
1.3	LibrettOS	21
1.3.1	Nombre del proyecto o sistema operativo	21
1.3.2	Enlace al repositorio y/o documentación oficial	21
1.3.3	Objetivo del proyecto	21
1.3.4	Lenguaje(s) de implementación	22
1.3.5	Arquitectura del sistema (monolítica, microkernel, etc.)	22
1.3.6	Componentes implementados (procesos, memoria, archivos, etc.)	23
1.3.6.1	Gestión de procesos y planificación	25
1.3.6.2	Gestion de memoria	25
1.3.6.3	Sistema de archivos	25
1.3.6.4	Controladores de dispositivo	25

1.3.6.5	Red	26
1.3.6.6	Servicios y compatibilidad POSIX	26
1.3.7	Herramientas utilizadas (compiladores, emuladores, etc.) . . .	26
1.3.8	Nivel de complejidad y accesibilidad para estudiantes	27
1.4	HelenOS	28
1.4.1	Nombre del proyecto o sistema operativo	28
1.4.2	Enlace al repositorio y/o documentación oficial	28
1.4.3	Objetivo del proyecto	28
1.4.4	Lenguaje(s) de implementación	28
1.4.5	Arquitectura del sistema (monolítica, microkernel, etc.)	29
1.4.6	Componentes implementados (procesos, memoria, archivos, etc.)	31
1.4.7	Herramientas utilizadas (compiladores, emuladores, etc.) . . .	35
1.4.8	Nivel de complejidad y accesibilidad para estudiantes	36
1.5	Haiku OS	37
1.5.1	Nombre del proyecto o sistema operativo	37
1.5.2	Enlace al repositorio y/o documentación oficial	37
1.5.3	Objetivo del proyecto	38
1.5.4	Lenguaje(s) de implementación	38
1.5.5	Arquitectura del sistema (monolítica, microkernel, etc.)	38
1.5.6	Componentes implementados (procesos, memoria, archivos, etc.)	40
1.5.7	Herramientas utilizadas (compiladores, emuladores, etc.) . . .	43
1.5.8	Nivel de complejidad y accesibilidad para estudiantes	44

2 Comparación técnica 45

3 Análisis crítico	46
Referencias	47

Índice de tablas

1.1	Funcionamiento técnico de los principales subsistemas de MINIX 3 . .	9
1.2	Comparación entre un kernel tradicional y el nanocore de Theseus . .	15
1.3	Funcionamiento técnico (idea clave) de los principales subsistemas de Theseus OS	17
1.4	Funcionamiento técnico de los principales subsistemas de LibrettOS .	24
1.5	Funcionamiento técnico (idea clave) de los principales subsistemas de HelenOS	32
1.6	Funcionamiento técnico (idea clave) de los principales subsistemas de HaikuOS	41

Índice de figuras

1.1	Arquitectura de MINIX 3.	8
1.2	Comparativa de la arquitectura de Theseus frente a arquitecturas tradicionales.	16
1.3	Arquitectura de LibrettOS.	23
1.4	Vista general de la arquitectura/organización del kernel de HelenOS.	30
1.5	Árbol de drivers en HelenOS.	34
1.6	Arquitectura del sistema operativo BeOS.	39

Introduccion

El desarrollo y la transformación de los sistemas operativos han sido fundamentales en la evolución de las tecnologías de la información. Desde los primeros programas creados para las computadoras centrales hasta los sistemas más utilizados en ordenadores personales, dispositivos móviles y equipos embebidos, los sistemas operativos han actuado como un puente entre las necesidades del usuario y la complejidad del hardware. Estudiarlos no solo permite entender el funcionamiento básico de una computadora, sino que también brinda las bases conceptuales y técnicas necesarias para el diseño de nuevas soluciones informáticas. Este documento tiene como finalidad ofrecer una perspectiva global de los sistemas operativos, abordando sus características principales, su arquitectura y sus componentes.

Objetivo de la investigacion

El objetivo principal de este trabajo es realizar un análisis técnico y comparativo de distintos sistemas operativos, con la finalidad de identificar sus características más relevantes y examinar las diversas concepciones y enfoques aplicados en su diseño e implementación. Se estudiarán tanto arquitecturas tradicionales, como la monolítica,

así como modelos más modernos, entre ellos los exokernels, híbridos y microkernels . Del mismo modo, se pretende ofrecer una visión crítica sobre las ventajas, limitaciones y ámbitos de aplicación de cada sistema operativo.

Alcance del análisis

El presente estudio se centra en sistemas operativos ampliamente utilizados y representativos de diferentes enfoques arquitectónicos y áreas de aplicación. No se abordarán sistemas altamente experimentales o versiones obsoletas a menos que sean relevantes para la comparación histórica o conceptual. El análisis incluye tanto sistemas de propósito general (Windows, macOS, Linux) como sistemas especializados (RTOS, embebidos y móviles), enfocándose en aspectos técnicos, operativos y comunitarios que permitan identificar fortalezas, limitaciones y escenarios de uso recomendados para cada sistema operativo.

Metodologia

Con el fin de alcanzar los objetivos planteados, se seguirá una metodología compuesta por tres fases fundamentales:

1. **Revisión bibliográfica y documental:** Se consultarán libros especializados, artículos académicos, blogs técnicos, documentación oficial y repositorios relacionados con los sistemas operativos en estudio.
2. **Evaluación técnica:** Se examinarán los aspectos esenciales de cada sistema, incluyendo la gestión de procesos, la administración de memoria, los sistemas de

archivos, el manejo de dispositivos, las interfaces de usuario y los mecanismos de seguridad.

3. **Comparación estructurada:** Se elaborará una tabla que sintetice las características más representativas de los sistemas operativos analizados, tomando en cuenta factores como el tamaño del kernel, la arquitectura, la comunidad, el lenguaje de implementación, la documentación disponible y el soporte de hardware.

Este enfoque permitirá garantizar un análisis más objetivo, completo y útil, tanto para fines académicos como prácticos.

Capítulo 1

Propuestas analizadas

1.1 MINIX

1.1.1 Nombre del proyecto o sistema operativo

El nombre del proyecto desarrollado por Andrew S. Tanenbaum es **MINIX**. Según (Andrew S. Tanenbaum and Albert S. Woodhull, 2006), este nombre es un acrónimo de *mini-UNIX*, su propósito fue ser una versión educativa y simplificada del sistema UNIX, diseñada para facilitar el aprendizaje de los principios fundamentales de los sistemas operativos. Este sistema se inspiró en la UNIX Version 7 y con el tiempo fue mejorado para ajustarse al estándar internacional POSIX.

1.1.2 Enlace al repositorio y/o documentación oficial

- Pagina oficial: <https://www.minix3.org>
- Enlace al repositorio oficial: <https://github.com/Stichting-MINIX-Research-Foundation/minix>
- Enlace al libro oficial: <https://csc-knu.github.io/sys-prog/books/Andrew%20S.%20Tanenbaum%20-%20Operating%20Systems.%20Design%20and%20Implementation.pdf>

1.1.3 Objetivo del proyecto

Segun (Andrew S. Tanenbaum and Albert S. Woodhull, 2006), este sistema operativo MINIX fue diseñado para ser un sistema operativo con fines educativos. Su propósito es ser como una herramienta de laboratorio para enseñar los conceptos fundamentales como: procesos, comunicación entre procesos, semáforos, monitores, paso de

mensajes, algoritmos de programación, entrada/salida, interbloqueos, controladores de dispositivos, gestión de memoria, algoritmos de paginación, diseño de sistemas de archivos, seguridad y mecanismos de protección. MINIX generalmente se centra en lo teórico y lo práctico es por esos motivos que fue uno de los mejores sistemas operativos para aprender sobre sistemas operativos.

1.1.4 Lenguaje(s) de implementación

Según (Andrew S. Tanenbaum and Albert S. Woodhull, 2006), MINIX fue implementado principalmente en el lenguaje de programación C, lo cual es típico para sistemas operativos debido a la conexión cercana con el hardware y la facilidad para escribir código. Durante años, algunas partes del código también han sido implementadas en ensamblador para aprovechar características específicas del hardware. En la versión de MINIX 3, tan solamente consta de 40000 líneas de código ejecutables, que lo hacen más fácil de entender y realizar modificaciones en el sistema operativo.

1.1.5 Arquitectura del sistema (monolítica, microkernel, etc.)

Según (Andrew S. Tanenbaum and Albert S. Woodhull, 2006, pag. 42–51), considera las siguiente modelo de estructuras de sistemas operativos.

- **Sistemas monolíticos:** En este sistema no tiene estructura, el sistema operativo está escrito como un conjunto de procedimientos, entonces cada cual puede hacer el llamado a cualquier procedimiento cuando lo necesite. Para construir el programa, primero se compila todos los procedimientos individuales, luego se unen en un solo archivo objeto. El sistema monolítico está dividido en 3 capas:

programa principal, procedimientos de servicio y procedimientos de utilidad.

- **Sistema en capas:** Es una mejora de modelo monolítico, donde este se organiza en un jerarquía de niveles, cada uno hecho sobre el inferior. Cada capa usa servicios a la capa inferior y realiza servicios a la capa superior
- **Maquinas virtuales** Por el motivo que los usuarios no tenían tiempo compartido. Se desarrollo sistemas de tiempo compartido. En este sistema se separo el multiprogramación y la maquina extendida, logrando crear varias maquinas virtuales idénticas al hardware real.
- **Exokernel:** Es un sistema creado por investigadores del MIT, que permite a cada usuario un clon de ordenador real, pero con una parte de todo lo recursos. funciona la mismo que el sistema de capas, pero en esta caso asigna y controla los recursos de las maquinas virtuales, Las ventaja es que elimina un capa de mapeo, ya no necesita reasignar direcciones de recurso, manteniendo en separación el multiprogramación y los sistemas de usuario.
- **Modelo cliente-servidor:** En este modelo se busca simplificar el sistema operativo trasladando la mayoría de sus funciones a procesos de usuario. Dejando un núcleo mínimo para la gestión de comunicación de procesos. Los clientes solicitan servicios y los servidores ejecutan y responden. Cada servidor se encarga de la memoria, procesos, archivos. A este sistema se le llama como modelo de sistemas distribuidos.

En el MINIX 3 se usa una arquitectura de microkernel, lo que nos dice que el núcleo o kernel se encarga de funciones mas básicas de sistema. como por ejemplo la plan-

ificación de procesos, comunicación, el manejo del hardware. Para poder entender mejor como es la arquitectura de MINIX 3, se muestra la siguiente figura 1.1.

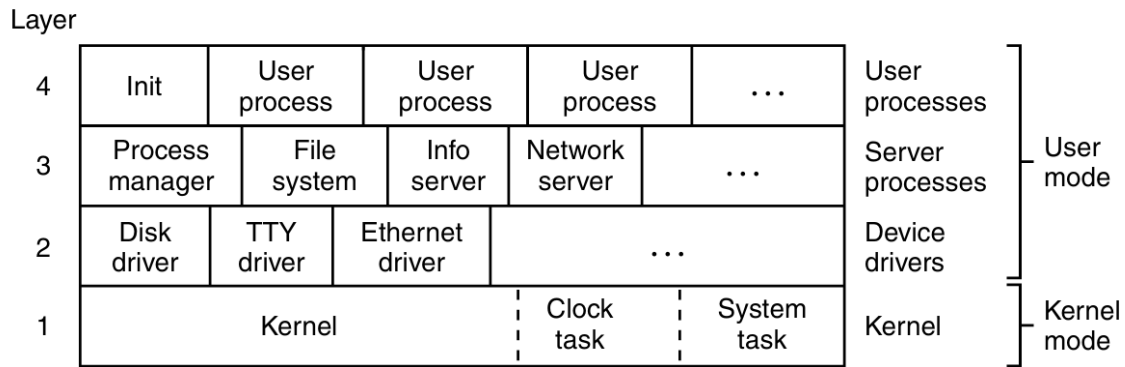


Figura 1.1: Arquitectura de MINIX 3.

Fuente: Obtenido de (Andrew S. Tanenbaum and Albert S. Woodhull, 2006, pag. 113)

OPERATING SYSTEMS DESIGN AND IMPLEMENTACIÓN.

1.1.6 Componentes implementados (procesos, memoria, archivos, etc.)

MINIX está diseñado para ser modular, donde cada servicio (como la gestión de archivos, la planificación de procesos y los controladores de dispositivos). En resumen podemos ver en la tabla 1.1 los principales subsistemas y su funcionamiento técnico.

Tabla 1.1: Funcionamiento técnico de los principales subsistemas de MINIX 3

Componente / Sub-sistema	Nombre en MINIX (archivo)	Funcionamiento técnico (idea clave)
Sistema de archivos	<code>fs/</code>	Gestiona la estructura de directorios y el montaje de dispositivos mediante <code>mount</code> .
Archivos especiales	<code>/dev/</code>	Representan dispositivos como archivos de bloque o carácter.
Tuberías (pipes)	<code>pipe.c</code>	Permiten comunicación entre procesos mediante pseudoarchivos.
Shell (intérprete de comandos)	<code>sh/</code>	Ejecuta programas, redirige E/S y permite multitarea con <code>&</code> .
Llamadas al sistema	<code>syscall.c</code>	Interfaz entre programas y el sistema operativo.
Comunicación entre procesos	<code>ipc.c</code>	Controla el intercambio seguro de mensajes entre procesos.

Fuente: Elaboración propia con base en (Andrew S. Tanenbaum and Albert S.

Woodhull, 2006, pag. 20–42).

1.1.6.1 Sistema de archivos

Gestiona la estructura jerárquica de archivos. Permite montar diferentes dispositivos (como CD-ROM, discos duros, disco solido, memoria flash y entre otros) en un único árbol de directorios mediante la llamada `mount`. Su trabajo es administra rutas,

directorios y accesos sin depender del dispositivo.

1.1.6.2 Archivos especiales

Modelan los dispositivos físicos como archivos. Los archivos de bloque permiten acceder a unidades de disco por bloques, mientras que los de carácter representan flujos continuos (impresoras, módems y todo dispositivos de entrada y salida). Con la finalidad de permitir operar los dispositivos usando las mismas llamadas que para archivos comunes.

1.1.6.3 Tuberías

Implementan la comunicación entre procesos mediante código en pseudoarchivos. Un proceso escribe en la tubería y otro lee, logrando sincronización y transferencia de datos sin necesidad de archivos temporales. Base del uso de tuberías en el shell.

1.1.6.4 Shell

Proporciona la interfaz principal con el usuario. Tiene como objetivo ejecutar programas, redirige entradas y salidas, conecta procesos con tuberías y permite tareas en segundo plano. Utiliza intensivamente las llamadas al sistema del núcleo.

1.1.6.5 Llamadas al sistema

Contiene la interfaz entre los programas de usuario y el sistema operativo. Incluyen operaciones para manejo de procesos y archivos. En MINIX 3, la mayoría de las llamadas siguen el estándar POSIX.

1.1.6.6 Comunicación entre procesos

Gestiona el intercambio de mensajes entre procesos del sistema y el microkernel. Esta arquitectura refuerza el aislamiento y la estabilidad, ya que los servicios del sistema operan como procesos independientes que se comunican de forma controlada.

1.1.7 Herramientas utilizadas (compiladores, emuladores, etc.)

MINIX acepta varios lenguajes de programación y compiladores, lo que facilita el desarrollo de software de usuario como compilaciones con el propio sistema operativo.

(The MINIX 3 Wiki, 2017)

- **Lenguajes:** Incluye soporte para varios lenguajes como C/C++, clisp(LISP), mawk(AWK), Perl, Python y otros.
- **Compiladores:** Utiliza el famoso gcc (GNU Compiler Collection) y clang/LLVM como compiladores. Ambos permiten la compilación nativa.
- **Arquitectura compatibles:** Está diseñado para procesadores x86, ARM y RISC-V.
- **Emuladores:** MINIX puede ejecutarse en emuladores populares como QEMU, VirtualBox y Bochs, lo que facilita su uso en diferentes entornos de desarrollo.
- **Paquetes:** Tiene una amplia variedad de programas, con más de 4000 paquetes disponibles, las cuales son Shells, editores de texto, juegos, correos electrónicos.

1.1.8 Nivel de complejidad y accesibilidad para estudiantes

Según (Andrew S. Tanenbaum and Albert S. Woodhull, 2006, pag. 17), MINIX 3 está especialmente enfocado en PCs más pequeños como los que se encuentran comúnmente en países en desarrollo y en sistemas integrados, que siempre tienen recursos limitados. En cualquier caso, este diseño facilita mucho a los estudiantes aprender cómo funciona un sistema operativo que intentar estudiar un sistema monolítico enorme. Por su reducido tamaño y diseño del micronúcleo y su documentación, resulta fácil de entender para personas que deseen instalar un sistema compatible con UNIX en sus máquinas personal (Colaboradores de Wikipedia, 2025).

1.2 Theseus

1.2.1 Nombre del proyecto o sistema operativo

El nombre del sistema operativo es “Theseus”; según (Theseus OS Project, 2023), fue nombrado así en honor a la paradoja del barco de Teseo, que plantea la cuestión de, si a un barco le renuevas todas sus piezas, ¿Ese barco renovado sigue siendo el mismo barco? Esta idea de renovación de cada componente es característica de este sistema operativo.

1.2.2 Enlace al repositorio y/o documentación oficial

- Enlace al repositorio oficial: <https://github.com/theseus-os/Theseus>
- Enlace al libro oficial: <https://www.theseus-os.com/Theseus/book/index.html>

1.2.3 Objetivo del proyecto

Según (Boos et al., 2020), este sistema operativo es en esencia experimental, aunque puede usarse como objeto de estudio debido a su innovadora arquitectura. También tiene objetivos técnicos, como reestructurar la modularidad, reducir el “state spill” y la recuperación ante fallos (fault recovery).

1.2.4 Lenguaje(s) de implementación

Según (Boos et al., 2020) y confirmando con el repositorio oficial (Boos, 2024), el sistema operativo Theseus fue casi completamente desarrollado en Rust, aunque también

se ha usado C para una futura implementación de la librería “libc” dirigida a Theseus (tlibc), ya que por el momento se tomó prestado de la biblioteca de REDOX (relibc).

1.2.5 Arquitectura del sistema (monolítica, microkernel, etc.)

Theseus, como sistema operativo no tiene una arquitectura clásica como monolítica o microkernel, ni mucho menos multikernel; sino que basa su estructura en **cells** o células por su traducción en español, las cuales haciendo honor a su definición biológica, son módulos pequeños e independientes que sirven como bloque de construcción para el sistema operativo. Adicionalmente, theseus tiene un componente mínimo el cual es llamado “nano-core”, que se encarga de iniciar el sistema, establecer memoria mínima y cargar las demás células; pero a diferencia de un kernel propiamente dicho, este no es un jefe permanente, sino que cada cell es independiente (Boos et al., 2020). A continuación una tabla comparativa entre un kernel clásico y el nano-core de theseus:

Tabla 1.2: Comparación entre un kernel tradicional y el nanocore de Theseus

Aspecto	Kernel tradicional	Nanocore (Theseus)
Definición	Es el núcleo del sistema operativo. Centraliza la gestión de componentes y recursos.	Unidad mínima del núcleo que gestiona sólo lo esencial para iniciar y mantener cells.
Estructura	Monolítica o microkernel: el kernel controla los servicios del sistema.	No existe un “núcleo único”: el sistema está compuesto por módulos cooperativos.
Acoplamiento	Alto. Los módulos dependen del kernel y de otros subsistemas.	Bajo. Cada módulo es reemplazable y se comunica mediante interfaces explícitas.
Objetivo	Proveer servicios centrales de manera estable.	Eliminar el <i>state spill</i> (derrame de estado), permite reemplazar componentes en ejecución.

Fuente: Adaptado en base a (Boos et al., 2020), *Theseus: An Experiment in OS Structure for State Management*.

Para entenderlo mejor, se especifica que se impone una arquitectura/organización plana para estas células, todo se corre en un solo SAS (single address space) y un solo nivel de privilegio (no se diferencia entre modo usuario y modo kernel) (Boos and Zhong, 2017). Además, se incluye una figura comparativa entre arquitecturas clásicas y la de theseus basada en células:

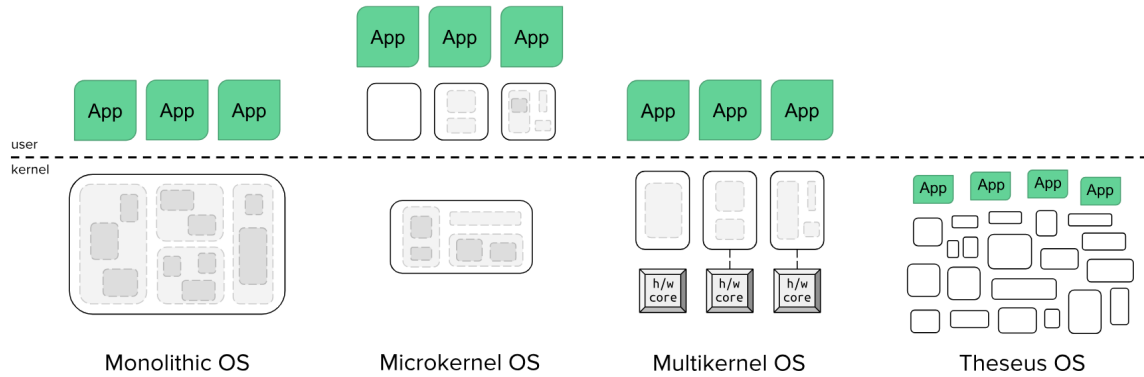


Figura 1.2: Comparativa de la arquitectura de Theseus frente a arquitecturas tradicionales.

Fuente: Obtenido de (Theseus OS Project, 2023) *The Theseus OS Book*.

1.2.6 Componentes implementados (procesos, memoria, archivos, etc.)

En theseus, cada cell encapsula una parte del sistema, encapsula cada componente, como gestión de memoria, planificación/Scheduling, E/S o comunicación. Este trabajo con cells permite aislar y detectar fallos, para poder reemplazar componentes en ejecución y evitar el *state spill* entre módulos (Boos et al., 2020). A continuación y a modo de introducción, se presenta una tabla con los principales componentes y subsistemas de Theseus OS, los nombres de sus archivos en el repositorio oficial y la idea clave detrás de sus funcionamiento.

Tabla 1.3: Funcionamiento técnico (idea clave) de los principales subsistemas de Theseus OS

Componente / Sub-sistema	Nombre en Theseus (archivo/crate)	Funcionamiento técnico (idea clave)
Gestión de memoria	<code>memory crate</code>	Mapea páginas virtuales a marcos físicos garantizando exclusividad.
Carga dinámica de módulos (módulos / células)	<code>mod_mgmt crate</code>	Carga en ejecución módulos (“cells”) y gestiona sus dependencias.
Scheduling / multitarea	<code>scheduler / task crates</code>	Ejecuta tareas en un único espacio de direcciones compartido.
Manejo de archivos / sistema de archivos	<code>fs crate</code>	Gestiona operaciones de archivos, bajo la modularidad de Theseus.
Subsistemas de E/S y drivers	Ej. <code>e1000</code> , <code>ata_pio</code> , <code>keyboard</code>	Controladores modulares escritos en Rust, diseñados para reemplazo.
Recuperación ante fallos / actualización en caliente	<code>loader</code> , <code>spawn crates</code>	Reemplaza módulos activos sin reiniciar el sistema completo.
Comunicación entre módulos	<code>event_types</code> , <code>device_manager</code> <code>crates</code>	Mensajería sin estado entre módulos para evitar dependencias implícitas.

Fuente: Elaboración propia con base en (Boos et al., 2020; Theseus OS Project, 2023; Boos and Zhong, 2017).

Gestión de memoria

El componente `memory` de Theseus implementa una idea `MappedPages`, que representa una región de páginas virtuales adyacentes, mapeadas a marcos físicos. El mapeo se realiza con la función `Mapper::map(pages, frames, flags, pg_tbl)`, y se aprovechan las utilidades de Rust para asegurar que cada página tenga un único marco asociado (Boos et al., 2020; Theseus OS Project, 2023).

Carga dinámica de módulos (módulos / células)

El subsistema `mod_mgmt` permite que los módulos (cells) se carguen, vinculen y descarguen en tiempo de ejecución. Para ello, se construyen instancias de `CrateNamespace`. Este subsistema nos permite sustituir un módulo (cell) sin reiniciar por completo el sistema (Theseus OS Project, 2023).

Scheduling / multitarea

Los crates `scheduler` y `task` gestionan las tareas en un único entorno de espacio de direcciones (SAS) y un solo nivel de privilegio (SPL). Cada tarea se encapsula mediante una estructura `Task` que contiene el contexto del CPU, el puntero de pila y el estado de ejecución. Este enfoque de unicidad evita el fenómeno del “state spill” entre módulos (Boos, 2024).

Manejo de archivos / sistema de archivos

El crate `fs` implementa las funciones básicas del sistema de archivos, como lectura, escritura y gestión de directorios. Siguiendo la filosofía de Theseus, este módulo

se carga como una cell independiente, permitiendo su reemplazo o actualización sin necesidad de reiniciar el sistema (Boos, 2024).

Subsistemas de E/S y drivers

Los drivers como `e1000`, `ata_pio` o `keyboard` escritos en Rust; gestionan hardware de red, discos PATA/IDE y teclados PS/2 respectivamente. Fiel al enfoque de Theseus, son independientes y reemplazables en tiempo de ejecución; para facilitar el “fault recovery” (Boos, 2024).

Recuperación ante fallos / actualización en caliente

Los crates `loader` y `spawn` son las cells que permiten el tan aclamado “fault recovery” y “live update”; o sea recuperación ante fallos y actualización, de cells, sin reiniciar todo el sistema. El funcionamiento contiene detención de tareas, liberación de recursos y recarga de cells (Theseus OS Project, 2023).

Comunicación entre módulos

Los subsistemas `event_types` y `device_manager` se encargan de la correcta mensajería entre cells mediante canales tipados y sin depender de estado compartido. Las dependencias se definen con metadatos para eliminar el “state spill” en la comunicación entre componentes (Boos and Zhong, 2017).

1.2.7 Herramientas utilizadas (compiladores, emuladores, etc.)

Theseus emplea y modifica herramientas de y para Rust; también utiliza máquinas virtuales. Las principales herramientas son las siguientes:

- **Rust toolchain:** Uso de `cargo`, `rustc` y `crates` estándar del ecosistema Rust para la compilación, gestión de dependencias y construcción modular de Theseus.
- **Sistema de carga dinámica y enlazado en tiempo de ejecución:** Theseus modifica el compilador (`rustc`) y el “linker” para permitir la carga y sustitución de (cells) en ejecución (Boos and Zhong, 2017).
- **Emuladores y máquinas virtuales:** Se utilizan entornos como “QEMU” para probar, depurar y validar el funcionamiento del sistema (Boos, 2024).
- **Librería estandar de C:** Actualmente, Theseus utiliza gran parte de `relibc` de Redox OS como su biblioteca estándar de C, pero planea desarrollar su propia versión llamada `tlibc` (Theseus lib) en el futuro (Boos, 2024).

1.2.8 Nivel de complejidad y accesibilidad para estudiantes

Considerando un entorno académico, que es justamente donde rust brilla, es un proyecto digno de estudio, con un altísimo nivel de complejidad debido también y en mayor medida, a su arquitectura basada en cells; también cabe mencionar que la dificultad sube debido al uso de la versión de la librería estandar de C (`relibc`) y el uso de QEMU, el cual es un emulador no tan intuitivo y del cual no hay tanta documentación en español.

Sin embargo, se ve compensado debido a su alta accesibilidad, la cual es posible gracias a su extensa documentación y estudios en torno a este proyecto, propiciados por el mismo creador de Theseus.

1.3 LibrettOS

1.3.1 Nombre del proyecto o sistema operativo

Segun la sitio oficial (Systems Software Research Group, 2021), El nombre de este proyecto es LibrettOS, un sistema operativo de código abierto desarrollado por el Grupo de Investigación de Software de Sistemas de Virginia Tech. Se basa en la combinación de dos paradigmas un microkernel y el modelo de biblioteca.

1.3.2 Enlace al repositorio y/o documentación oficial

- Pagina oficial: <https://librettos.org/>
- Enlace al repositorio oficial: <https://github.com/ssrg-vt/librettos-src>
- Artículo: <https://ssrg.ece.vt.edu/papers/vee20-librettos.pdf>

1.3.3 Objetivo del proyecto

Según (Ruslan Nikolaev and Mincheol Sung and Binoy Ravindran, 2021), LibrettOS es un sistema operativo experimental de investigaciones. Su objetivo es explorar el paradigma del núcleo híbrido que combine con el microkernel multiserver, para mantener mayor seguridad y tolerancia a errores a fallos y permitir a las aplicaciones el acceso a directamente al hardware para un alto rendimiento. Específicamente no fue creado para fines educativo ni para uso constante, sino como un prototipo academico para diseñar nuevas ideas acerca de sistemas operativos.

1.3.4 Lenguaje(s) de implementación

Según el repositorio (The rumpkernel project, 2025), LibrettOS fue implementado principalmente en el lenguaje de programación C (55.8%), C++(0.5%), Assembly(9.5%), Makefile(4.2%) y objective-C(0.3%), lo cual es clásico para sistemas operativos debido a la conexión cercana con el hardware y la facilidad para escribir código. Durante años, algunas partes del código también han sido implementadas en ensamblador para aprovechar características específicas del hardware.

1.3.5 Arquitectura del sistema (monolítica, microkernel, etc.)

Según (Ruslan Nikolaev and Mincheol Sung and Binoy Ravindran, 2021, pag. 5–6), la arquitectura de LibrettOS combina modos de ejecución para optimizar el rendimiento y el aislamiento. La aplicación 1 se ejecuta en el modo de sistema operativo de biblioteca, accediendo a todos los dispositivos de hardware. La aplicación 2 se comunica a través de servidores de red. LibrettOS se ejecuta sobre Xen, un hypervisor de la arquitectura microkernel, el sistema permite varios accesos directos como indirectos al hardware. Para el almacenamiento, permite compartir archivos mediante un servidor NFS, actualmente LibrettOS es compatible con API POSIX/BSD. Para entender mejor la arquitectura, se muestra en la figura 1.3.

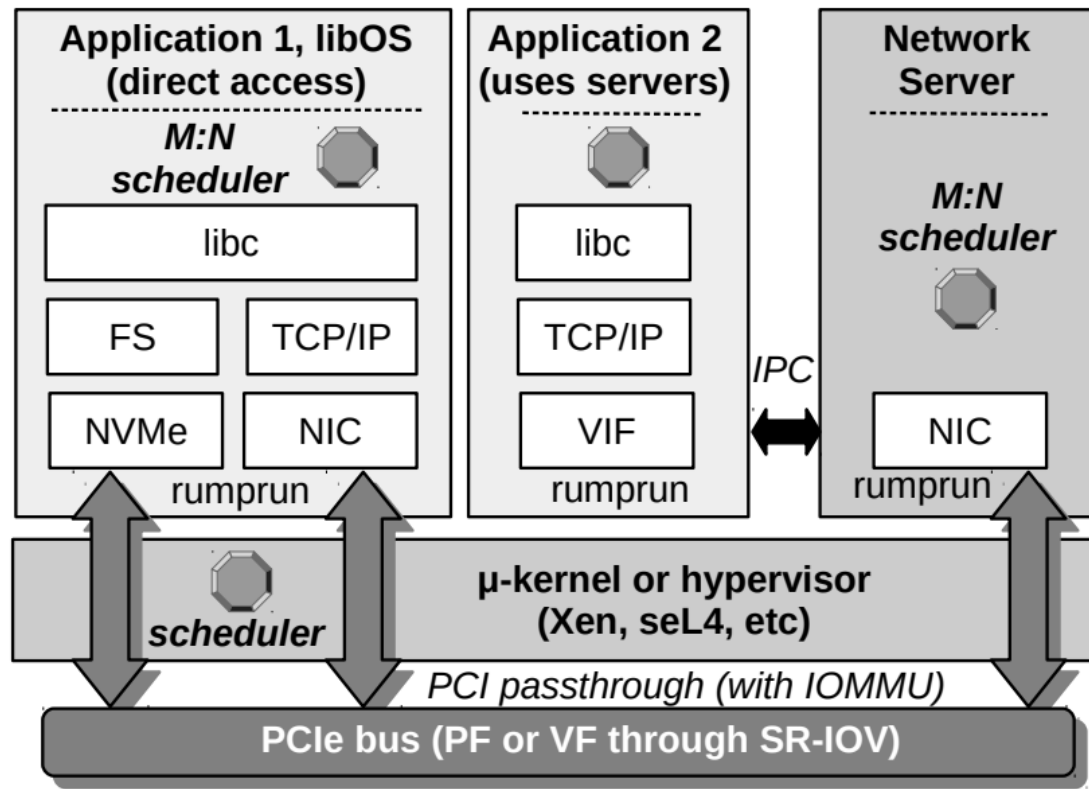


Figura 1.3: Arquitectura de LibrettOS.

Fuente: Obtenido de (Ruslan Nikolaev and Mincheol Sung and Binoy Ravindran, 2021, pag. 5) *LibrettOS: A Dynamically Adaptable Multiserver-Library OS*.

1.3.6 Componentes implementados (procesos, memoria, archivos, etc.)

LibrettOS es un sistema modular que distribuye los servicios del sistema operativo (procesos, memoria, archivos, red y controladores) entre servidores de usuario y librerías en espacio de aplicación. La tabla 1.4 resume sus principales subsistemas y su funcionamiento técnico.

Tabla 1.4: Funcionamiento técnico de los principales subsistemas de LibrettOS

Componente / Sub-sistema	Implementación en LibrettOS	Funcionamiento técnico (idea clave)
Gestión de procesos y planificación	<code>rumpkernel</code> / <code>rumprun-smp</code>	Cada dominio ejecuta un <i>rumpkernel</i> multi-hilo con planificación M:N. Xen gestiona los vCPUs y Rumprun programa los hilos POSIX.
Gestión de memoria	NetBSD VM / Xen / IOMMU	Usa memoria virtual de NetBSD con soporte para <i>PCI passthrough</i> , <i>IOMMU</i> y <i>SR-IOV</i> para aislar y compartir hardware.
Sistema de archivos	<code>ext3</code> / NFS / NVMe	Emplea <code>ext3</code> sobre NVMe y un servidor NFS en Rumprun para compartir volúmenes por red con buen rendimiento.
Controladores de dispositivos	NetBSD <code>drivers</code> / <code>rumprun-smp</code>	Reutiliza drivers de NetBSD (NIC, NVMe), operando igual en modo aislado o <i>library-OS</i> .
Red (Network stack)	NetBSD TCP/IP / servidor de red VIF	Ejecuta la pila TCP/IP en la app o un servidor de red, usando canales virtuales (VIF) para el reenvío seguro.
Servicios y compatibilidad POSIX	<code>rumprun</code> / NetBSD <code>libc</code>	Soporta servicios POSIX/BSD comunes; algunas llamadas como <code>fork()</code> aún no están implementadas.

Fuente: Elaboración propia con base en (Ruslan Nikolaev and Mincheol Sung and

Binoy Ravindran, 2021).

1.3.6.1 Gestión de procesos y planificación

Gestiona el intercambio de mensajes entre procesos del sistema y el microkernel. Esta arquitectura refuerza el aislamiento y la estabilidad, ya que los servicios del sistema operan como procesos independientes que se comunican de forma controlada.

1.3.6.2 Gestión de memoria

Cada dominio maneja su propia memoria virtual usando los mecanismos estándar de NetBSD sobre Rumprun. Para acceso directo a hardware usa PCI passthrough e IOMMU, lo que permite apartamiento seguro entre dominios. Además, soporta SR-IOV para dividir dispositivos físicos NICs o NVMe en funciones virtuales asignables a cada VM.

1.3.6.3 Sistema de archivos

Reutiliza el sistema de archivos de NetBSD. Una instancia de Rumprun con un volumen ext3 sobre NVMe actúa como servidor NFS, exportando volúmenes por red. Los clientes montan este recurso mediante NFS, demostrando buen rendimiento en pruebas de E/S.

1.3.6.4 Controladores de dispositivo

Reutiliza los controladores de NetBSD como por ejemplo, ixgbe para NICs Intel 10GbE y NVMe. Los mismos drivers funcionan en ambos modos de operación como para aislado o library-OS, permitiendo cambiar dinámicamente de modo sin reemplazar controladores.

1.3.6.5 Red

Ejecuta la pila TCP/IP de NetBSD dentro del espacio de aplicación o en un servidor de red dedicado. Este servidor reenvía tramas L2 mediante canales virtuales VIF. Si el servidor falla, el estado TCP se mantiene en la aplicación, lo que permite su recuperación.

1.3.6.6 Servicios y compatibilidad POSIX

LibrettOS es compatible con POSIX/BSD, soportando servicios como SSH, bases de datos y utilidades estándar. Algunas llamadas complejas como `fork()` y `pipe()`, aún no están implementadas, pero se consideran alcanzables en futuras versiones.

1.3.7 Herramientas utilizadas (compiladores, emuladores, etc.)

Según (Ruslan Nikolaev and Mincheol Sung and Binoy Ravindran, 2021), LibrettOS se basa en tecnologías como rump kernels y el uso de POSIX.

- **Rump kernels:** Utilizados para ejecutar sistemas operativos como *anykernels* fuera del kernel monolítico de NetBSD.
- **Rumprun:** Un unikernel basado en *rump kernels* para ejecutar aplicaciones directamente con menos sobrecarga del sistema operativo.
- **Hipervisores como Xen y KVM:** Usados para gestionar máquinas virtuales.
- **IOMMU y PCI Passthrough:** Herramientas para dar acceso directo a dispositivos físicos desde VMs.

- **DPDK y SPDK:** Bibliotecas de alto rendimiento para el acceso directo a hardware en aplicaciones de red y almacenamiento.

1.3.8 Nivel de complejidad y accesibilidad para estudiantes

Según el repositorio (The rumpkernel project, 2025), describen que LibrettOS es un prototipo experimental y advierte en un nota que no esta diseñado para uso diario normal, usar bajo su propia responsabilidad. Entonces esto quiere decir que no esta diseñado como un proyecto educativo fácil de usar, sino como una plataforma de investigación. Por lo tanto su dificultad esta alta, pero si es accesible a toda el código mediante el repositorio y a la web oficial de LibrettOS.

1.4 HelenOS

1.4.1 Nombre del proyecto o sistema operativo

El nombre del sistema operativo es “HelenOS”. El proyecto fue iniciado por Jakub Jermar en 2001, en ese momento era un código independiente que funcionaba como kernel para IA-32. (Děcký, 2015)

1.4.2 Enlace al repositorio y/o documentación oficial

- Enlace al repositorio oficial: <https://github.com/HelenOS/helenos>
- Documentación oficial y Wiki: <https://www.helenos.org/>

1.4.3 Objetivo del proyecto

Según (Děcký, 2015), este sistema operativo, tiene un enfoque educativo y experimental. Además, busca servir como una plataforma para la investigación y desarrollo de sistemas operativos de propósito general, teniendo muy en cuenta la fiabilidad y practicidad.

1.4.4 Lenguaje(s) de implementación

Según (Děcký, 2010) y confirmando con el repositorio oficial (Jermář, 2025), el sistema operativo HelenOS está principalmente implementado en C, con algo de apoyo de C++, Meson como “build system”, algunos componentes escritos en ensamblador para tareas de bajo nivel y python para scripts auxiliares.

1.4.5 Arquitectura del sistema (monolítica, microkernel, etc.)

El sistema operativo HelenOS se basó en la arquitectura multiserver de microkernel, multiserver es un tipo de arquitectura microkernel, donde el núcleo del sistema operativo (microkernel) es responsable de las funciones básicas para el funcionamiento del sistema operativo como gestión de memoria, comunicación de procesos y scheduling; y por otro lado, los demás servicios del sistema operativo se implementan como “servidores” y se ejecutan en el espacio de usuario. De esta manera, los servidores pueden ser desarrollados, cambiados y reiniciados de manera independiente sin afectar al kernel (Děcký, 2015). A continuación, se incluye una vista general de la arquitectura/organización del kernel de HelenOS:

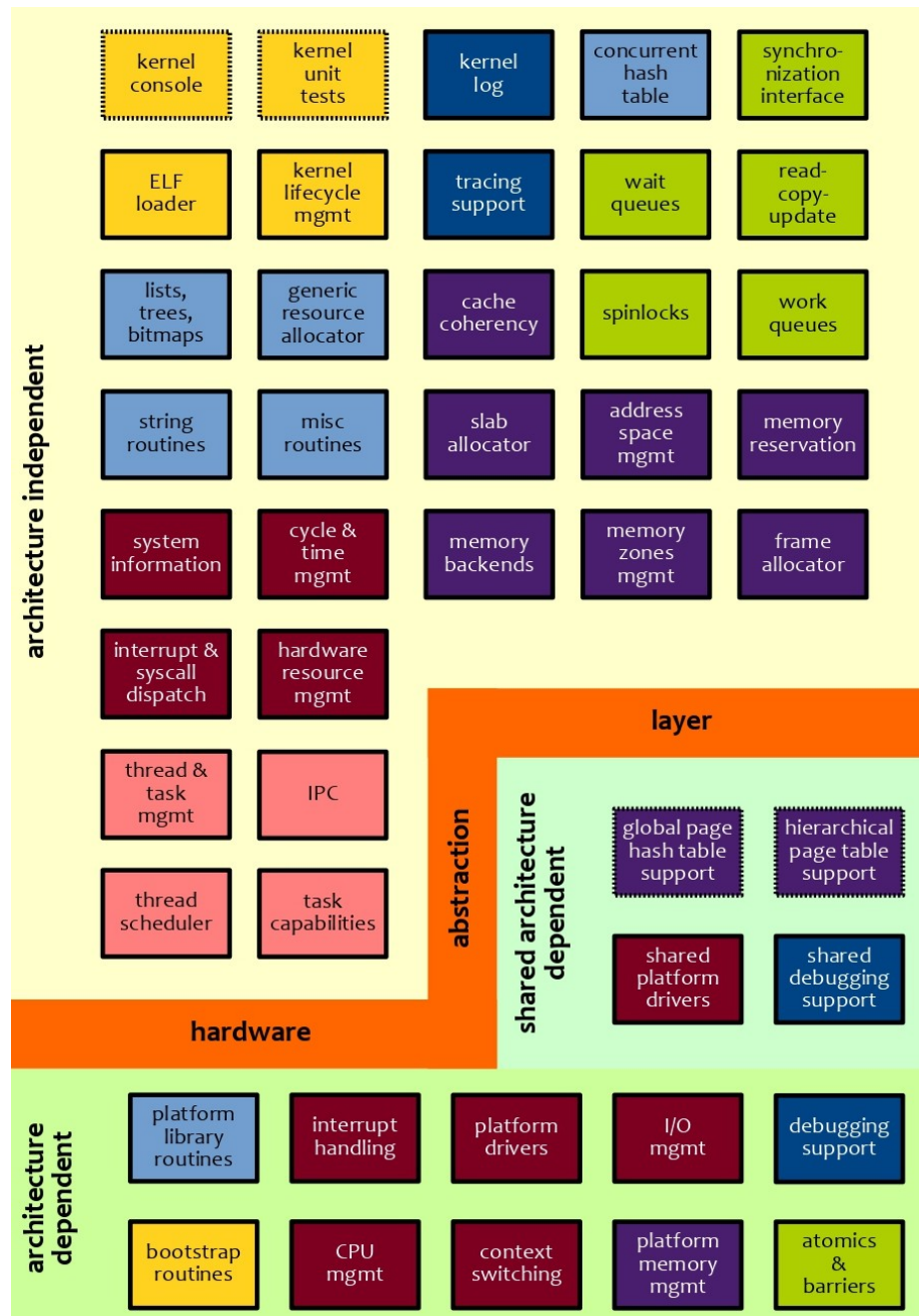


Figura 1.4: Vista general de la arquitectura/organización del kernel de HelenOS.

Fuente: Obtenido de (Děcký, 2015) *Application of Software Components in Operating System Design*.

1.4.6 Componentes implementados (procesos, memoria, archivos, etc.)

En HelenOS, cada componente del sistema operativo se implementa como una tarea aislada con su propio espacio de direcciones (siguiendo la arquitectura multiserver), identificador único y comunicación mediante interfaces bien definidas (IPC o syscall API) (Děcký, 2015). A continuación se presenta una tabla con los principales componentes y subsistemas de HelenOS, su nombre de archivo en el repositorio oficial y la idea técnica detrás de su implementación.

Tabla 1.5: Funcionamiento técnico (idea clave) de los principales subsistemas de HelenOS

Componente / Sub-sistema	Nombre en HelenOS (tarea / servicio)	Funcionamiento técnico (idea clave)
Gestión de memoria	frame allocator, slab allocator, address space mgmt	Asignación de marcos físicos con separación por tareas.
Planificación / multitarea	thread scheduler, task mgmt	Aislamiento por espacio de direcciones y comunicación por IPC.
Sistema de archivos	TMPFS, FAT, ext4, MINIX FS	Implementados como tareas independientes, comunicadas vía VFS.
Subsistemas de E/S y drivers	Ej. rtl8139, ahci, ps2, usb hid	Se usan como tareas separadas, organizados en el árbol de dispositivos.
Servicios de red	inetsrv, tcp, udp, ethip	Componentes aislados, comunicados por IPC.
Servicios de usuario / consola	console, clipboard, compositor, bdsh	Tareas independientes con comunicación por IPC.
Servicios de nombres y localización	naming service, location service	Servicios singleton para identificar y vincular componentes.

Fuente: Elaboración propia con base en (Děcký, 2015; Jermář, 2025).

Gestión de memoria

En HelenOS, se implementan los componentes `frame allocator`, `slab allocator` y `address space management` para asignar marcos físicos, gestionar zonas de memoria y mantener espacios de direcciones separados por tarea. Esta separación es para encapsular el estado de cada componente y facilitar la verificación de corrección (Děcký, 2015, p. 39).

Planificación / multitarea

La planificación se realiza mediante `thread scheduler` y `task management`. Le generan a cada tarea su propio espacio de direcciones y permiten su comunicación con otras mediante IPC (Děcký, 2015, p. 38).

Sistema de archivos

HelenOS soporta múltiples sistemas de archivos como `TMPFS`, `FAT`, `ext4`, `MINIX FS`, cada uno implementado como una tarea independiente; se comunican con el sistema mediante el `vfs`, el cual es un intermediario entre drivers y aplicaciones (Děcký, 2015, p. 40).

Subsistemas de E/S y drivers

Los drivers están organizados jerárquicamente en el árbol de dispositivos (como se muestra en la figura). Cada driver (como `rtl8139`, `ahci`, `ps2`, `usb hid`) se implementa como una tarea separada, para permitir modularidad, aislamiento y reemplazo en ejecución (Děcký, 2015, p. 41).

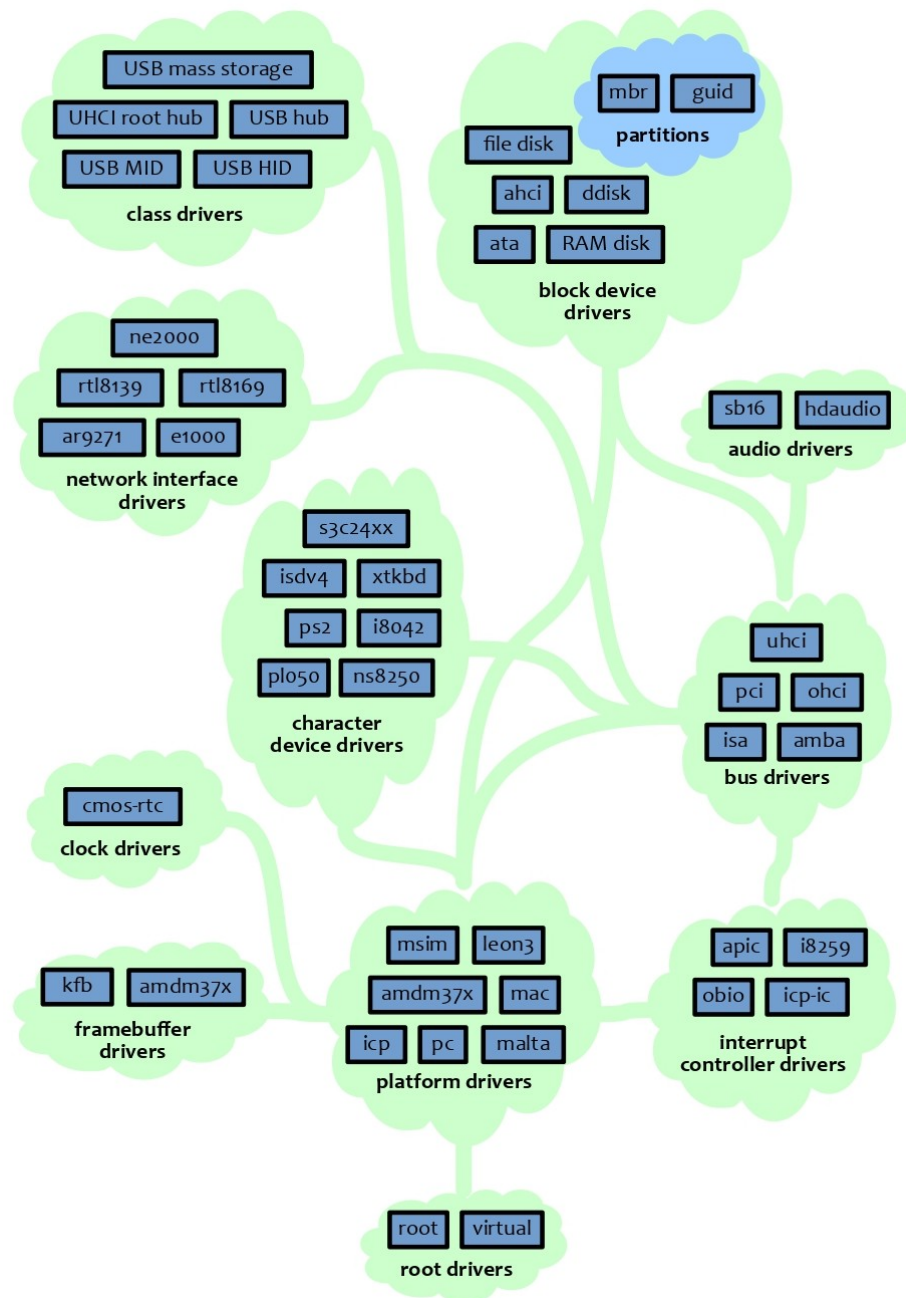


Figura 1.5: Árbol de drivers en HelenOS.

Fuente: Obtenido de (Děcký, 2015) *Application of Software Components in Operating System Design*.

Servicios de red

Los protocolos de red `tcp`, `udp`, `ethip` y el servicio `inetsrv` se implementan como componentes independientes. Siguiendo así la arquitectura multiserver (Děcký, 2015, p. 40).

Servicios de usuario / consola

Los componentes `console`, `clipboard`, `compositor` y el shell `bdsh` se ejecutan como tareas separadas (Děcký, 2015, p. 40).

Servicios de nombres y localización

Son servicios singleton, específicamente `naming service` y el `location service`, que permiten identificar y vincular componentes en el sistema (Děcký, 2015, p. 38).

1.4.7 Herramientas utilizadas (compiladores, emuladores, etc.)

HelenOS utiliza herramientas para el lenguaje C, también usa máquinas virtuales, sistemas de construcción e implementaciones. Entre sus principales herramientas están:

- **Compilador:** Uso de GCC y Clang, con toolchains cruzadas para las diferentes arquitecturas soportadas (x86, ARM, MIPS, SPARC, PowerPC e Itanium) (Jermář, 2025).
- **Emuladores y máquinas virtuales:** Para ejecutar HelenOS se usa el entorno “QEMU”, para probar, depurar y validar el ISO del microkernel (Jermář, 2025).

- **Sistema de construcción:** Makefile con scripts Bash, Python y uso de MESON (experimental) para apoyar la compilación cruzada de componentes (Jermář, 2025).
- **Herramienta de portabilidad:** *HelenOS Coastline*, un repositorio desarrollado por los creadores del microkernel, puede ser usado para la integración de software externo más fácilmente.

1.4.8 Nivel de complejidad y accesibilidad para estudiantes

Considerando un entorno de estudio de sistemas operativos, justamente el entorno en el que se propicia esta investigación, con conocimientos previos de C y fundamentos de sistemas operativos; es un proyecto con un nivel de complejidad media, más que nada por su tamaño (es extenso), además del uso de servicios mediante IPC y también considerar que la arquitectura multiserver, que implica microkernel, no es la más sencilla tampoco. Y eso sin contar la diversidad de arquitecturas soportadas, lo cual añade dificultad al análisis del código.

Se consideró en la categorización del nivel de complejidad que, la accesibilidad para los estudiantes es bastante buena, ya que el proyecto es open source y cuenta con una gran cantidad de estudios en torno a él. Sin embargo, una consideración a tomar en cuenta es que, aunque HelenOS dispone de documentación formal (guías, artículos, documentación generada), esta puede no detallar cada componente del sistema operativo con la exhaustividad que otros sistemas operativos grandes ofrecen (por ejemplo un libro completo o wiki ultra-detallada).

1.5 Haiku OS

1.5.1 Nombre del proyecto o sistema operativo

Según (Ryan Leavengood, 2012), Haiku Os es un proyecto visionario con el nombre del sistema operativo Haiku OS en la actualidad, pero que antes era OpenBeOS, un sistema operativo fundado en la División de Corporaciones del Estado de Nueva York como una organización sin fines de lucro en julio del 2003. por su fundador Michael Phipps. El nombre del sistema operativo proviene de la palabra japonesa “Haiku“, por que era usada en los mensajes de error de BeOS, eran mensajes presentados de forma poética japonesa.

1.5.2 Enlace al repositorio y/o documentación oficial

- **Repositorio oficial (espejo en GitHub):** <https://github.com/haiku/haiku> — funciona como un *mirror* del repositorio principal alojado en el servidor de Haiku, donde se gestiona el desarrollo activo del sistema operativo. (Haiku Project)
- **Repositorio principal (Gerrit):** <https://review.haiku-os.org/haiku> — servidor oficial utilizado para la revisión y aprobación del código fuente. (Haiku Project)
- **Sitio web oficial del proyecto:** <https://www.haiku-os.org/get-haiku/installation-guide/>

1.5.3 Objetivo del proyecto

Según (Stimac, Miroslav, 2011) Haiku es un proyecto comunitario orientado al escritorio experimental, como meta ofrecer un sistema operativo de escritorio abierto, ligero y fácil de usar, diseñado específicamente para la computación personal inspirado en BeOS, Haiku se creó con la finalidad de recrear experiencia de uso para entender el funcionamiento y realizar mejoras.

1.5.4 Lenguaje(s) de implementación

Haiku está implementado en C++, haciendo uso de una API de Be orientada a objetos (The kits) que facilita el desarrollo de aplicaciones y controladores de dispositivos (Ryan Leavengood, 2012). Incluye contenedores de la Biblioteca de Plantillas Estándar de C++ (STL) y otras bibliotecas de terceros como OpenSSL, SQLite y zlib (Haiku, Inc., 2025).

1.5.5 Arquitectura del sistema (monolítica, microkernel, etc.)

El sistema operativo está basado en BeOS un Kernel híbrido modular. Su núcleo se desarrolló de una rama de NewOS, la modularidad permite a los controladores y otros componentes que se carguen de manera dinámica todo el proceso según sea necesario. Las siguientes son las capas principales que conforman el sistema operativo, según (Nakrani, Jatinkumar, 2025):

- **Capa de núcleo:** El núcleo de NewOS sirve para servicio como: multitarea, gestión de memoria, controladores y administrador de archivos.

- **Servidor de aplicaciones:** El servidor de aplicaciones gestiona las ventanas, el dibujo y el renderizado. Esta diseñado para un alto rendimiento y se integra con el servidor de entrada y el kit de herramientas.
- **Gestión de paquetes:** Haiku es compatible con Be File System (BFS) que incluye atributos de archivos extendidos.
- **Controlador de red:** Haiku incluye una pila de red modular compatible con estaciones.

En la siguiente imagen es al arquitectura del sistema operativo BeOS (el precursor de HaikuOS). Se puede observar que entre el hardware y las aplicaciones del software se encuentra el software BeOS. El software del sistema operativo tiene 3 capas, capa de micronúcleo, capa de servidor y capa de software.

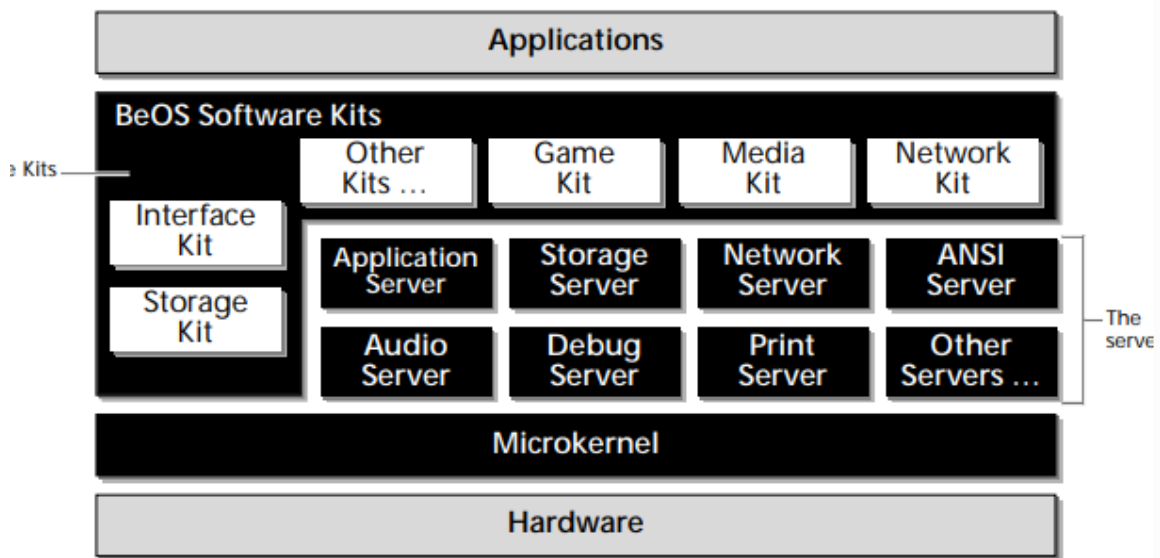


Figura 1.6: Arquitectura del sistema operativo BeOS.

Fuente: Obtenido de (Be, Inc., 2025)

1.5.6 Componentes implementados (procesos, memoria, archivos, etc.)

En HaikuOS, como es característico en sistemas operativos de arquitectura híbrida modular, los componentes principales están implementados y se ejecutan en modo kernel, pero con una alta modularidad organizada en “kits” y servidores. A continuación se presenta una tabla con los principales componentes y subsistemas de HaikuOS, su ubicación en el repositorio oficial y la idea técnica detrás de su implementación.

Tabla 1.6: Funcionamiento técnico (idea clave) de los principales subsistemas de HaikuOS

Componente / Sub-sistema	Nombre / Ruta en repositorio	Funcionamiento técnico (idea clave)
Gestión de memoria	src/system/kernel/vm/, paging.cpp	Asignar páginas, espacio de direcciones por proceso.
Scheduling / multitarea	src/system/kernel/ thread.cpp, scheduler.cpp	Soporte para multiprocesador, planificación preventiva y cambio de contexto eficiente.
Sistema de archivos	src/add-ons/kernel/ file_systems/	BFS extendido, con journalling; soporte FAT para compatibilidad.
Drivers / E/S	src/add-ons/kernel/ drivers, bus_managers/	Implementados como módulos independientes y jerarquizados.
Servicios de red	src/add-ons/kernel/ network/, src/servers/net_server/	Pila TCP/IP modular, soporte para sockets BSD, DHCP y servicios de red como servidores.
Interfaz gráfica / servicios de usuario	src/servers/app/, src/servers/input/, src/kits/interface/, app_server, input_server	Servidores para gestión gráfica y entrada, comunicación mediante kits orientados a objetos (GUI toolkit).

Fuente: Elaboración propia con base en (Project, 2025; Nakrani, Jatinkumar, 2025).

Gestión de memoria

En HaikuOS se gestiona la memoria virtual con asignación de páginas bajo demanda, con método indexado y vector de bits para administrar espacio libre; y cada proceso tiene su propio espacio de direcciones. El módulo de “Virtual Memory Manager” se encarga de la asignación física y lógica de memoria (Gonzales Calos Alejandro, Lozano Eliseo de Jesús, 2017).

Scheduling / multitarea

La planificación de hilos en el sistema operativo HaikuOS soporta más de un núcleo. Cada hilo tiene su propio contexto; el scheduler organiza prioridades (para usar el CPU) y cambios de contexto (Nakrani, Jatinkumar, 2025).

Sistema de archivos

En HaikuOS, se tiene el sistema de archivos BFS (Be File System) con metadatos extendidos (pares clave-valor) y búsquedas indexadas. Además tiene soporte para otros sistemas como FAT para compatibilidad (Nakrani, Jatinkumar, 2025; Project, 2025).

Drivers / E/S

Los drivers están organizados en forma de árbol con E/S (la cual está muy optimizada para contenido multimedia) en forma de módulos independientes, para permitir actualizarse en ejecución (Haiku, Inc., 2025; Be, Inc., 2025).

Servicios de red

El sistema operativo HaikuOS, cuenta con un kit (Network Kit) muy completo y el servidor `net_server` para comunicaciones. Tiene soporte para los protocolos TCP/IP y UDP, incluso, si le añades complementos, AppleTalk o IPX (Nakrani, Jatinkumar, 2025; Be, Inc., 2025).

Interfaz gráfica / servicios de usuario

La interfaz gráfica de usuario de Haiku es gestionada por los servidores `app_server` e `input_server`. Los “kits” de desarrollo (Application, Interface) nos proporcionan APIs con el paradigma orientado a objetos en C++ para aplicaciones y servicios (Nakrani, Jatinkumar, 2025).

1.5.7 Herramientas utilizadas (compiladores, emuladores, etc.)

HaikuOS utiliza diversas herramientas para ejecutar, compilar y construir. También usa máquinas virtuales e implementaciones con funcionalidades adicionales. Entre las principales herramientas se encuentran:

- **Compilador:** Uso de GCC 2 (modificado), para mantener compatibilidad con BeOS, y GCC 13 para el desarrollo actual (Yusuf, Mohamad and Al Fatah, M Reza and Fami, Asrul and Pratama, Vemas Adi and Z, M Fauzan and Syadam, Muhammad, 2024).
- **Emuladores y máquinas virtuales:** En (Yusuf, Mohamad and Al Fatah, M Reza and Fami, Asrul and Pratama, Vemas Adi and Z, M Fauzan and Syadam, Muhammad, 2024) se hace uso de VirtualBox y en su página oficial (Haiku

Project, 2025) se recomienda usar QEMU, así que HaikuOS puede ejecutarse en ambos entornos.

- **Sistema de construcción:** Haiku modifica la herramienta “Jam” para compilar el kernel, los drivers y la interfaz gráfica. También ofrece compilación cruzada para otras arquitecturas (Haiku Project, 2025).
- **Herramientas de portabilidad:** En el repositorio (Project, 2025), se encuentran scripts *raw disk*, *MMC* o *SD card* para generar imágenes en diferentes arquitecturas.

1.5.8 Nivel de complejidad y accesibilidad para estudiantes

Considerando un entorno académico de estudio de sistemas operativos, justamente el ambiente para el cual se propicia esta investigación, con conocimientos previos de C (el antecesor de C++) y fundamentos de sistemas operativos; HaikuOS es un proyecto con nivel de complejidad media, debido a su arquitectura híbrida modular, la cual no es muy común, y a la cantidad de componentes implementados, y la profundidad de los mismos (por ejemplo, la interfaz gráfica y optimización para multimedia). Además de la herramientas utilizadas para su construcción y portabilidad.

También se consideró para colocarlo en el nivel medio, que la accesibilidad para los estudiantes es buena, ya que el proyecto es open source, aunque no cuenta con tantos estudios en torno a él como otros sistemas. Su página web oficial es bastante completa, y el repositorio oficial está bien organizado. Sin embargo, no posee una documentación formal tan detallada como otros sistemas operativos.

Capítulo 2

Comparación técnica

La tabla siguiente presenta una comparación técnica de varios sistemas operativos destacados, considerando aspectos clave como arquitectura, lenguaje de programación, tamaño del kernel, soporte de hardware, documentación y comunidad de usuarios y desarrolladores.

Capítulo 3

Análisis crítico

Referencias

Andrew S. Tanenbaum and Albert S. Woodhull (2006). *Operating Systems: Design and Implementation*. Prentice Hall, 3rd edition. Accedido el 13 de octubre de 2025.

Be, Inc. (2025). The BeOS Bible: BeOS Kits. Espejo (mirror) de la documentación oficial de BeOS. Documentación original ca. 1998-2000. Accedido el 31 de octubre de 2025.

Boos, K. (2024). Theseus — source code (github repository). <https://github.com/theseus-os/Theseus>. Repositorio accedido el 22 Octubre 2025.

Boos, K., Liyanage, N., Ijaz, R., and Zhong, L. (2020). Theseus: an experiment in operating system structure and state management. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1–19. USENIX Association.

Boos, K. and Zhong, L. (2017). Theseus: a state spill-free operating system. In *Proceedings of the ACM Workshop on Programming Languages and Operating Systems (PLOS '17)*. ACM.

Colaboradores de Wikipedia (2025). MINIX. Wikipedia, la enciclopedia libre. Página

modificada por última vez el 29 de septiembre de 2025. Accedido el 23 de octubre de 2025.

Děcký, M. (2010). A road to a formally verified general-purpose operating system. In *Proceedings of the International Symposium on Architecting Critical Systems (ISARCS '10), Lecture Notes in Computer Science, vol. 6150*. Recuperado el 22 Octubre 2025.

Děcký, M. (2015). *Application of Software Components in Operating System Design*. PhD thesis, Charles University in Prague, Faculty of Mathematics and Physics.

Gonzales Calos Alejandro, Lozano Eliseo de Jesús (2017). Administración e instala sistemas operativos Haiku. Presentación en Scribd. Subido el 21 de septiembre de 2017. Accedido el 31 de octubre de 2025.

Haiku, Inc. (2025). Haiku. Sitio web oficial. Copyright 2001-2025. Accedido el 23 de octubre de 2025.

Haiku Project (2025). Haiku — The Open Source Operating System. Sitio web oficial. Accedido: 20 septiembre 2025.

Jermář, J. (2025). Helenos — source code (github repository). <https://github.com/HelenOS/helenos>. Repositorio accedido el 22 Octubre 2025.

Nakrani, Jatinkumar (2025). Deployment of Android, Haiku, and Google Fuchsia in Containers and Virtual Machines with Web-UI Access via Apache Guacamole. Tesis de Maestría, Frankfurt University of Applied Sciences. Accedido el 31 de octubre de 2025.

Project, T. H. (2025). Haiku — source code (github repository). <https://github.com/haiku/haiku>. Repositorio accedido el 22 de octubre de 2025.

Ruslan Nikolaev and Mincheol Sung and Binoy Ravindran (2021). LibrettOS: A Dynamically Adaptable Multiserver-Library OS. In *Proceedings of the 2021 USENIX Annual Technical Conference (ATC '21)*. Accedido el 23 de octubre de 2025.

Ryan Leavengood (2012). The Dawn of Haiku OS. *IEEE Spectrum*. Accedido el 23 de octubre de 2025.

Stimac, Miroslav (2011). The desktop operating system Haiku®: Analysis of the operating system with focuses on ease of use, GUI, multimedia capability and an empirical research of the Haiku community. Tesis de Maestría, FernUniversität in Hagen (University of Hagen). Accedido el 1 de noviembre de 2025.

Systems Software Research Group (2021). LibrettOS: A research operating system. Sitio web oficial del proyecto, Virginia Tech. Copyright 2021. Accedido el 23 de octubre de 2025.

The MINIX 3 Wiki (2017). MINIX 3 Features. MINIX 3 Official Wiki. Página modificada por última vez el 27 de junio de 2017. Accedido el 23 de octubre de 2025.

The rumpkernel project (2025). rumprun: The rumprun unikernel and toolstack. Repositorio oficial en GitHub. Accedido el 23 de octubre de 2025.

Theseus OS Project (2023). The theseus os book. Recuperado de <https://www.theseus-os.com/Theseus/book/index.html>.

Yusuf, Mohamad and Al Fatah, M Reza and Fami, Asrul and Pratama, Vemas Adi and Z, M Fauzan and Syadam, Muhammad (2024). Installing the Haiku Operating System on Virtual Box and Implementing File Management. *Journal Collabits*, 1(2). Accedido el 31 de octubre de 2025.